

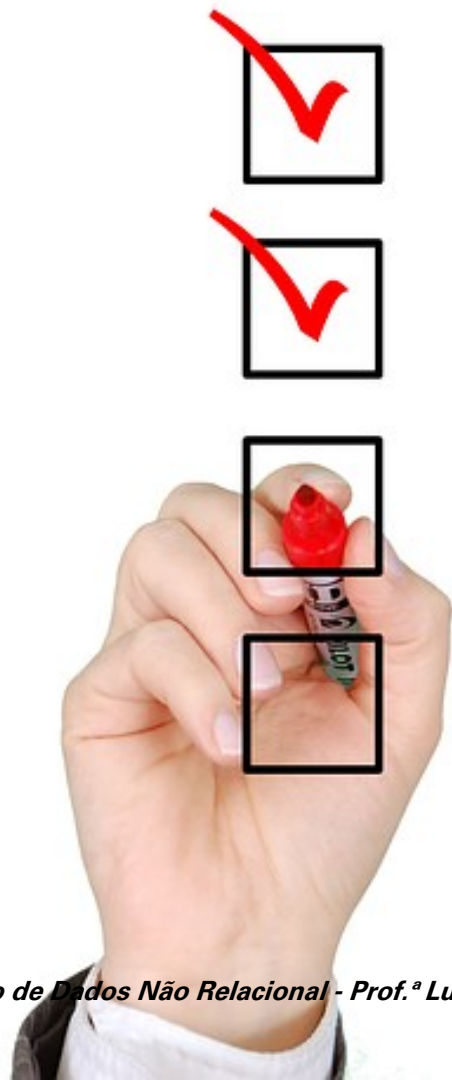
BANCO DE DADOS NÃO RELACIONAL

Monitoramento, Backup e Segurança no MongoDB

Professora:

Lucineide Pimenta

Tópicos da Aula



O que vamos aprender:

- ❑ Compreender como proteger os dados e o acesso ao MongoDB.
- ❑ Aprender estratégias de backup e restauração.
- ❑ Utilizar ferramentas de monitoramento e boas práticas de segurança.
- ❑ Aplicar esses conceitos no contexto do projeto ABP.

Por que Segurança é Importante?

- ❑ O **MongoDB** armazena dados sensíveis.
- ❑ Sem controle de acesso, qualquer pessoa pode ler ou excluir informações.
- ❑ Segurança envolve:
 - ❑ **Autenticação** (quem é o usuário)
 - ❑ **Autorização** (o que ele pode fazer)
 - ❑ **Criptografia** (proteger os dados)
 - ❑ **Backups** (recuperar informações em caso de falha)

Autenticação no MongoDB

A autenticação identifica **quem acessa o banco**.

Exemplo – Criar um usuário administrador:

```
use admin
db.createUser({
  user: "admin",
  pwd: "123456",
  roles: [ { role:
"userAdminAnyDatabase", db: "admin"
} ]
})
```

Ativar autenticação no servidor:

- 1- Editar o arquivo [mongod.conf](#).
- 2- Adicionar:
security:
[authorization: enabled](#)
- 3- Reiniciar o MongoDB.

Autenticação na Conexão Node.js

```
mongoose.connect("mongodb://admin:123456@127.0.0.1:27017/estacao_meteorologica?authSource=admin")
```

Explicação:

- ❑ **authSource** define em qual banco o usuário está registrado.
- ❑ Sempre use variáveis de ambiente (**.env**) para esconder a senha.

Arquivo .env

Exemplo:

```
MONGO_URI=mongodb://admin:123456@127.0.0.1:27017/estacao_meteorologica?authSource=admin  
PORT=3000
```

Leitura no Node.js:

```
require('dotenv').config();  
mongoose.connect(process.env.MONGO_URI);
```

Controle de Acesso por Funções (Roles)

Funções (roles) determinam o que o usuário pode fazer.

Exemplo:

```
db.createUser({  
  user: "meteorologista",  
  pwd: "senha123",  
  roles: [ { role: "readWrite", db: "estacao_meteorologica" } ]  
})
```

- ❑ Esse usuário poderá **ler e escrever**, mas não criar outros usuários nem excluir bancos.

Boas Práticas de Segurança

- ✓ Crie usuários com **permissões mínimas necessárias**.
- ✓ Nunca use o usuário admin em aplicações.
- ✓ Use variáveis de ambiente para credenciais.
- ✓ Atualize o MongoDB regularmente.
- ✓ Habilite firewall para restringir IPs de acesso.

Criptografia de Dados

- ❑ O MongoDB oferece **criptografia em repouso (at rest)** e **em trânsito (TLS/SSL)**.
- ❑ No ambiente local, é possível simular com **bibliotecas no Node.js**.
- ❑ **Exemplo simples:**

```
const crypto = require("crypto");  
const texto = "Senha123";  
const hash = crypto.createHash("sha256").update(texto).digest("hex");  
console.log(hash);
```

Backup no MongoDB

Backup com Mongodump

- ❑ Ferramenta oficial para backup do MongoDB.
- ❑ **Comando:**
mongodump --db estacao_meteorologica --out C:\backups
- ❑ Cria uma cópia dos dados em formato binário.

Restauração com Mongorestore

- mongorestore --db estacao_meteorologica C:\backups\estacao_meteorologica*
- ❑ Recupera os dados salvos.

Backup no MongoDB

Backup em JSON

- ❑ Para exportar dados legíveis:

```
mongoexport --db estacao_meteorologica --collection leituras --out leituras.json
```

Restauração com JSON

```
mongoimport --db estacao_meteorologica --collection leituras --file leituras.json
```

Agendando Backups Automáticos

- ❑ **Dica:** usar o **Node.js** com **node-cron**.

```
const cron = require("node-cron");
```

```
const { exec } = require("child_process");
```

```
cron.schedule("0 2 * * *", () => {
```

```
  exec("mongodump --db estacao_meteorologica --out ./backup", (err) => {
```

```
    if (err) console.error("Erro no backup", err);
```

```
    else console.log("Backup diário realizado!");
```

```
  });
```

```
});
```

- ❑ Executa o backup todos os dias às 2h da manhã.

Visualizações de acesso

Usando mongostat

mongostat --host 127.0.0.1 --rowcount 5

- ❑ Mostra: conexões, consultas por segundo, leituras e escritas.

Logs do MongoDB

- ❑ Localizados em */var/log/mongodb/mongod.log*.
- ❑ Permitem rastrear erros e operações.
- ❑ Sempre revise logs após falhas de conexão ou queda de performance.

Monitoramento no MongoDB

Monitoramento com Mongoose

```
mongoose.connection.on("connected", () => console.log("Banco conectado"));  
mongoose.connection.on("disconnected", () => console.log("Banco desconectado"));  
mongoose.connection.on("error", err => console.error("Erro:", err));
```

Backup e Segurança no Projeto ABP

- ❑ Cada grupo deve:
 - ❑ Criar backup do banco da ABP.
 - ❑ Demonstrar restauração em máquina diferente.
 - ❑ Documentar o processo no relatório final.

Boas Práticas

- ✓ Fazer backups diários.
- ✓ Armazenar cópias em nuvem (Google Drive, AWS, etc.).
- ✓ Monitorar logs e conexões.
- ✓ Testar restauração periodicamente.

Erros Comuns

- × Usar usuário admin em aplicações.
- × Deixar porta 27017 aberta sem autenticação.
- × Não testar backups.
- × Ignorar logs de erro.

BANCO DE DADOS NÃO RELACIONAL

Atividade Prática (Individual)

Material de apoio

- ❑ MongoDB Security Checklist
- ❑ MongoDB Backup & Restore Docs
- ❑ Node-cron Documentation

Bibliografia Básica

- ❑ BOAGLIO, Fernando. **MongoDB**: Construa novas aplicações com novas tecnologias. São Paulo: Casa do Código, 2015.
- ❑ ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**: Fundamentos e Aplicações. 7ed. São Paulo: Pearson, 2019.
- ❑ SADALAGE, P.; FOWLER, M. **Nosql Essencial**: Um Guia Conciso Para o Mundo Emergente da Persistência Poliglota. São Paulo: Novatec, 2013.
- ❑ SINGH, Harry. **Data Warehouse**: conceitos, tecnologias, implementação e gerenciamento. São Paulo: Makron Books, 2001.

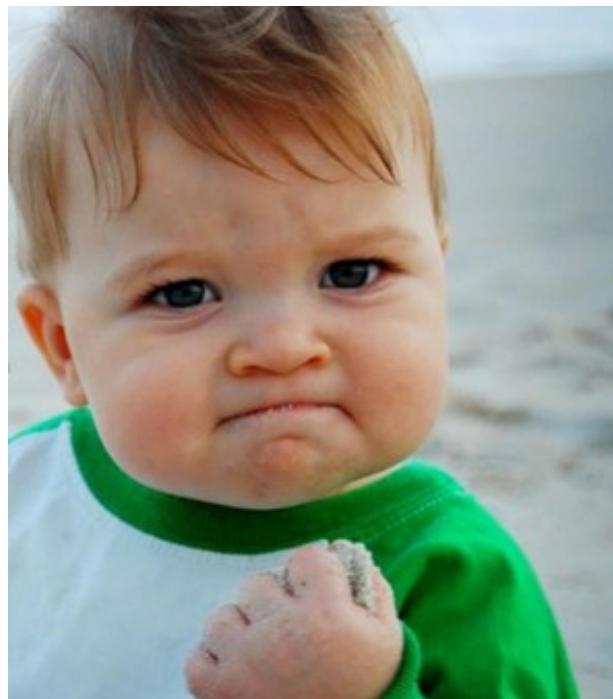
Bibliografia Complementar

- ❑ FAROULT, Stephane. **Refatorando Aplicativos SQL**. Rio de Janeiro: Alta Books, 2009.
- ❑ PANIZ, D. **NoSQL**: Como armazenar os dados de uma aplicação moderna. Casa do Código, 2016.
- ❑ SOUZA, M. **Desvendando o MongoDB**. Rio de Janeiro: Ciência Moderna, 2015.

Dúvidas?



Considerações Finais



**Professora:
Lucineide Pimenta**

Bom descanso à todos!

